

Start Trek

Results Report - Reinforcement Learning Lunar Lander

This report presents the results obtained with three reinforcement learning agents on LunarLander-v3: DQN, Double DQN and Double DQN with Prioritized Experience Replay. The training graphs are based on 500 episodes. The final evaluation was run over 100 episodes for each trained model.

1. Evaluation Summary

The main metric for the final comparison is the mean reward over 100 evaluation episodes. A score above 200 is considered a solved environment. The standard deviation is important because it shows how stable the agent is. A high mean with a lower deviation is better than a high mean with very unstable episodes.

Agent	Evaluation result	Solved?	Training landing rate	Interpretation
DQN	195.10 +/- 87.23	No, but close	55.6%	The agent learned a useful policy, but it remains unstable and finishes just below the solved threshold.
Double DQN	241.42 +/- 52.89	Yes	62.8%	Best final result. It passes the threshold and has the lowest variance among the three models.
Double DQN + PER	225.59 +/- 68.01	Yes	67.8%	Best landing rate during training, but final evaluation is less stable than Double DQN.

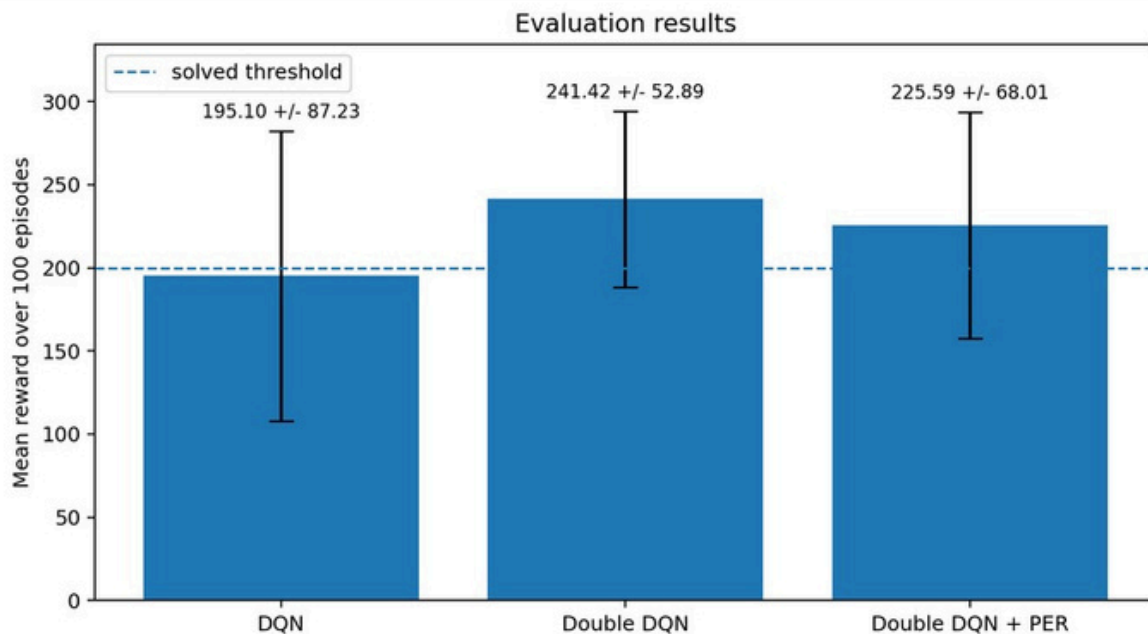


Figure 1: Final evaluation scores over 100 episodes. Error bars show the standard deviation.

Main conclusion

Double DQN is the best final model in this experiment. It reaches the highest evaluation score and the lowest standard deviation. Double DQN + PER lands more often during training, but its final evaluation score is lower and more variable. DQN is a good baseline because it clearly learns, but it does not consistently reach the solved threshold during evaluation.

2. Experimental Setup

The project uses the discreteLunarLander-v3 environment from Gymnasium. The agent observes the position, velocity, angle, angular velocity and leg contacts of the lander, then chooses one of four actions: do nothing, fire the left thruster, fire the main engine or fire the right thruster.

During training, the code logs the reward, episode length, epsilon value, loss and termination causes. The termination causes used in the graphs are landed, crash, sleep and out_of_view. These values help explain not only whether the agent gets a high score, but also why episodes end.

Element	Value
Environment	Gymnasium LunarLander-v3, discrete control
Training length	500 episodes for each agent
Evaluation length	100 episodes for each trained model
Compared agents	DQN, Double DQN, Double DQN + PER
Main success metric	Mean reward over 100 evaluation episodes
Solved threshold	Mean reward greater than or equal to 200

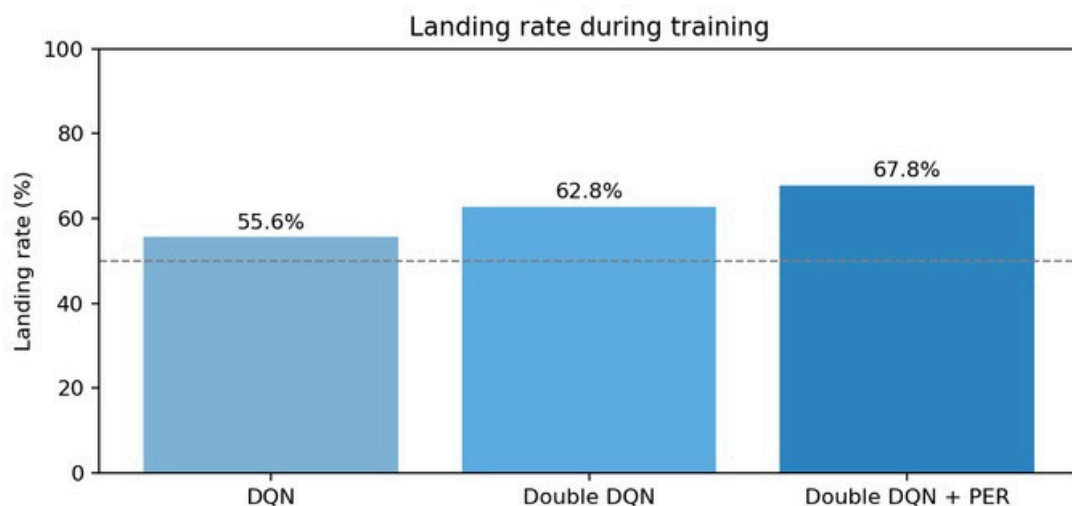


Figure 2: Landing rate during the 500 training episodes for each agent.

How the results should be read

The reward curve shows if learning improves. The 50-episode mean is more useful than the raw reward because the raw curve is noisy. The episode length gives context: long episodes can mean the lander survives longer, but they can also mean that it is waiting or drifting without landing. The loss curve should generally decrease, but reinforcement learning losses are not expected to be perfectly smooth.

3. DQN Results

The DQN agent shows a clear learning progression. The reward starts mostly negative, then the moving average improves and reaches the area around the solved threshold near the end of training. This means that the agent learned how to control the lander instead of behaving randomly.

The final evaluation gives 195.10 $\pm$ 87.23 over 100 episodes. This is a good result for a baseline because it is very close to 200, but it is not enough to validate the environment as solved. The standard deviation is also high, meaning that some episodes are successful while others still fail badly.

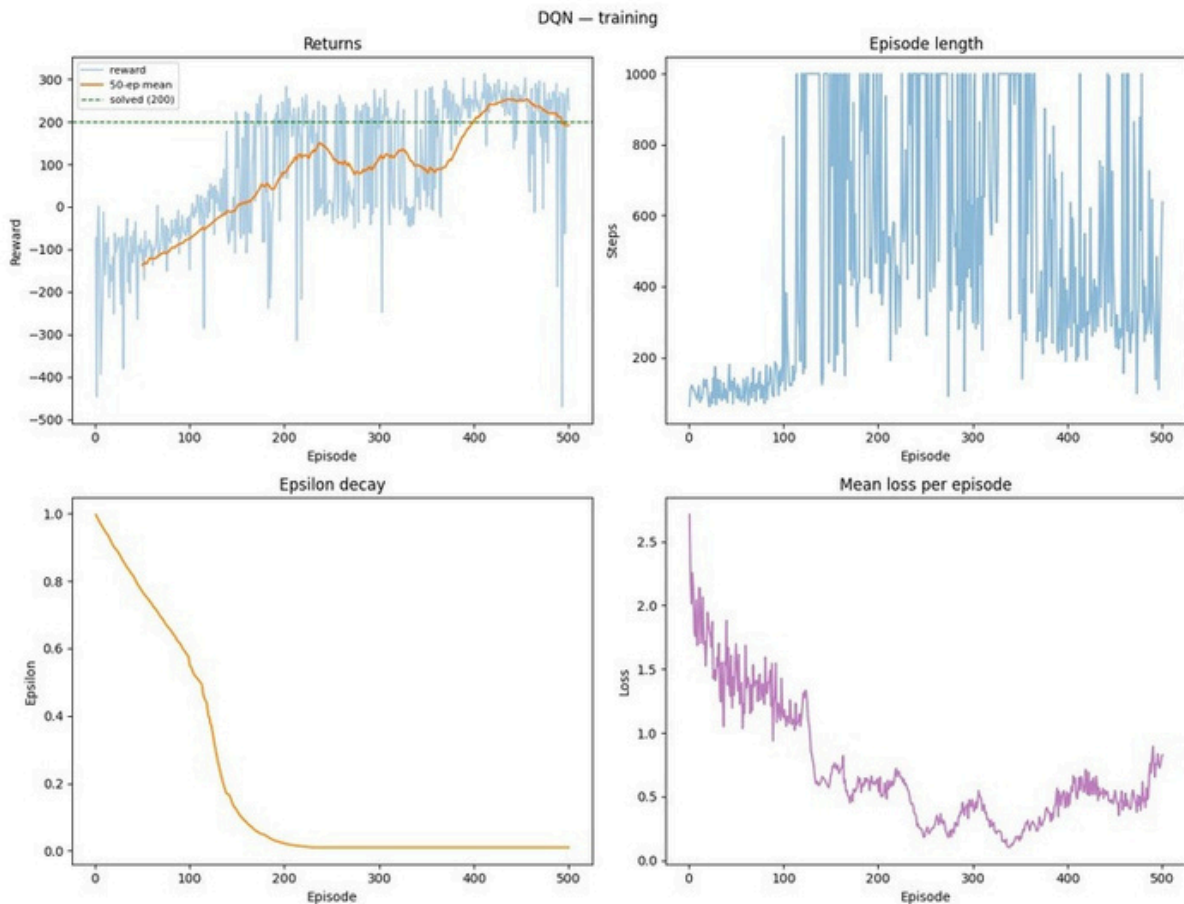


Figure 3: DQN training reward, episode length, epsilon decay and loss.

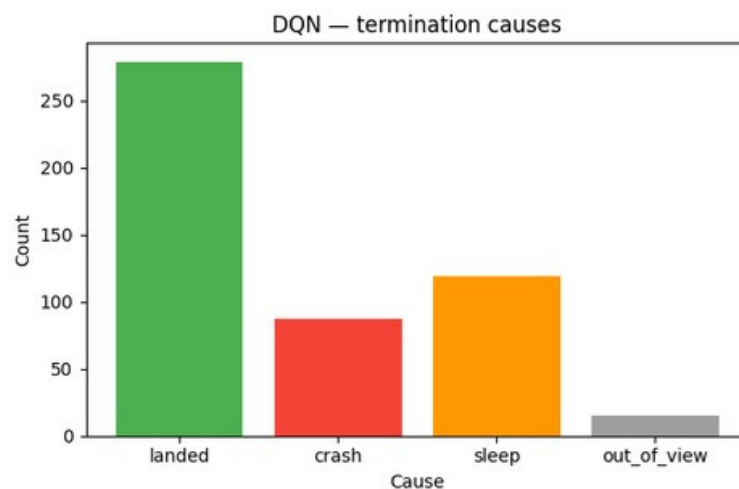


Figure 4: DQN termination causes during training.

The termination graph confirms this instability. The agent lands many times, but there are still a significant number of crashes and sleep endings. This explains why the mean score is close to the target but the variance remains high.

4. Double DQN Results

DoubleDQN gives the strongest finalevaluation result. Its training curve improves quickly and the moving average goes above the solved threshold near the end. Compared with regular DQN, the final reward is higher and the evaluation is more stable.

The final evaluation gives 241.42 +/- 52.89 over 100 episodes. This is the best result of the three agents. The mean reward is clearly above 200, and the standard deviation is the lowest. This means the model does not only land well sometimes, but does it more consistently.

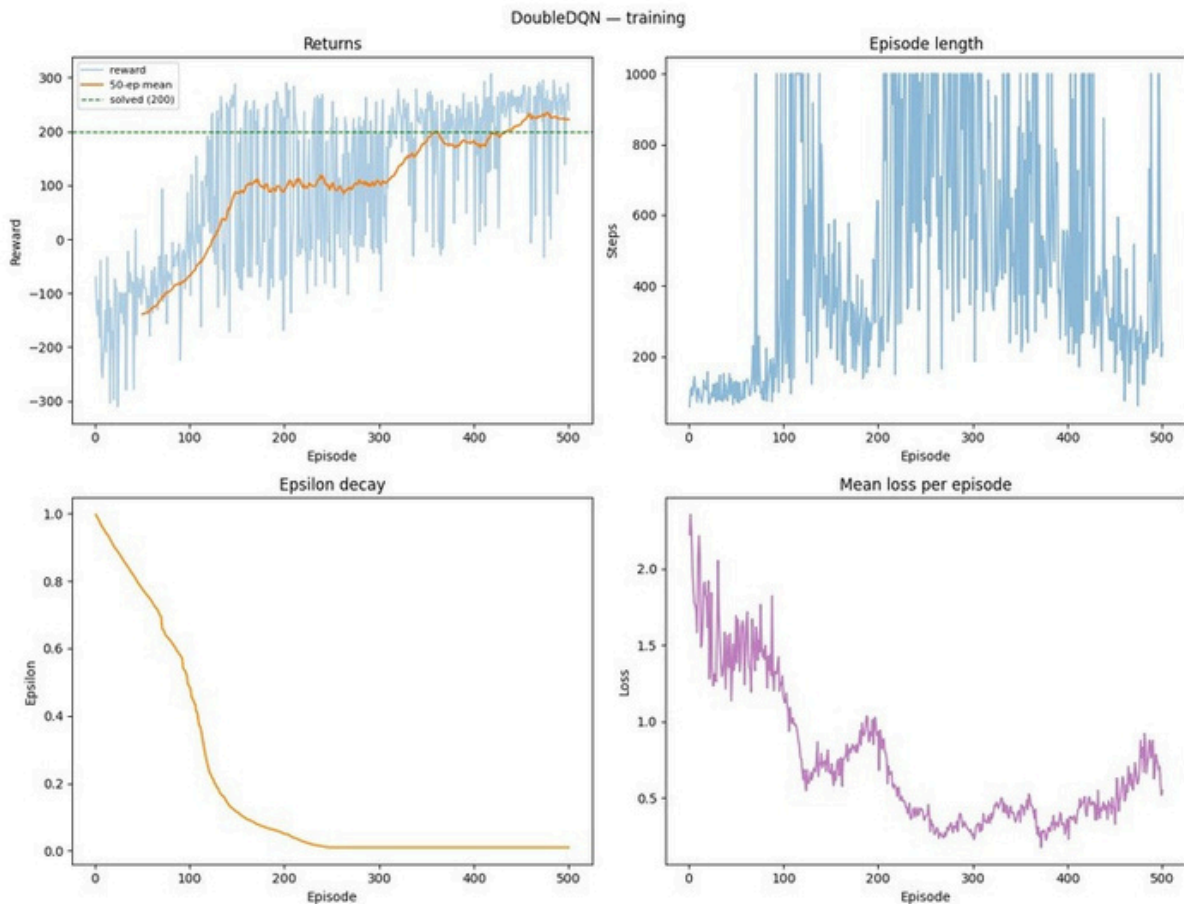


Figure 5: Double DQN training reward, episode length, epsilon decay and loss.

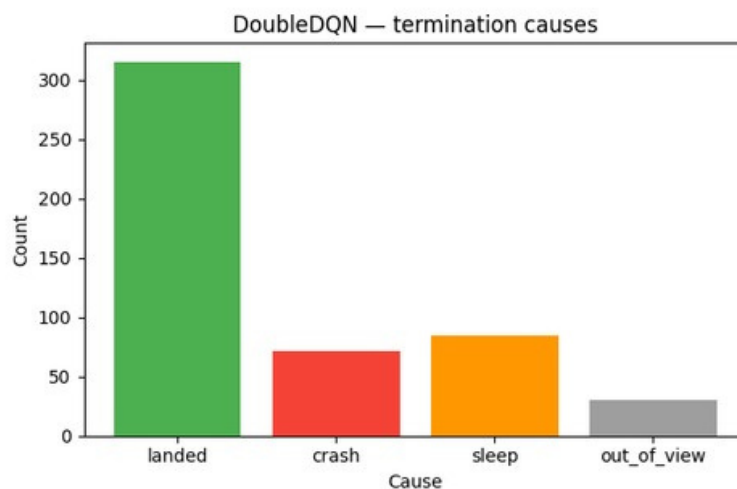


Figure 6: Double DQN termination causes during training.

The result is good because Double DQN usually reduces the overestimation problem of classic DQN. In this project, that seems to help the agent select better actions and obtain a more reliable landing behavior.

5. Double DQN + PER Results

Double DQN with Prioritized Experience Replay has the best landing rate during training. It reaches 67.8% landings over the 500 training episodes, which is higher than both DQN and Double DQN. The training curve also shows a strong final performance, with the moving average staying above the solved threshold near the end.

The final evaluation gives 225.59 +/- 68.01 over 100 episodes. This is a good result because the mean reward is above 200, so the agent passes the target. However, it is not the best final model because its mean score is lower than Double DQN and its standard deviation is higher.

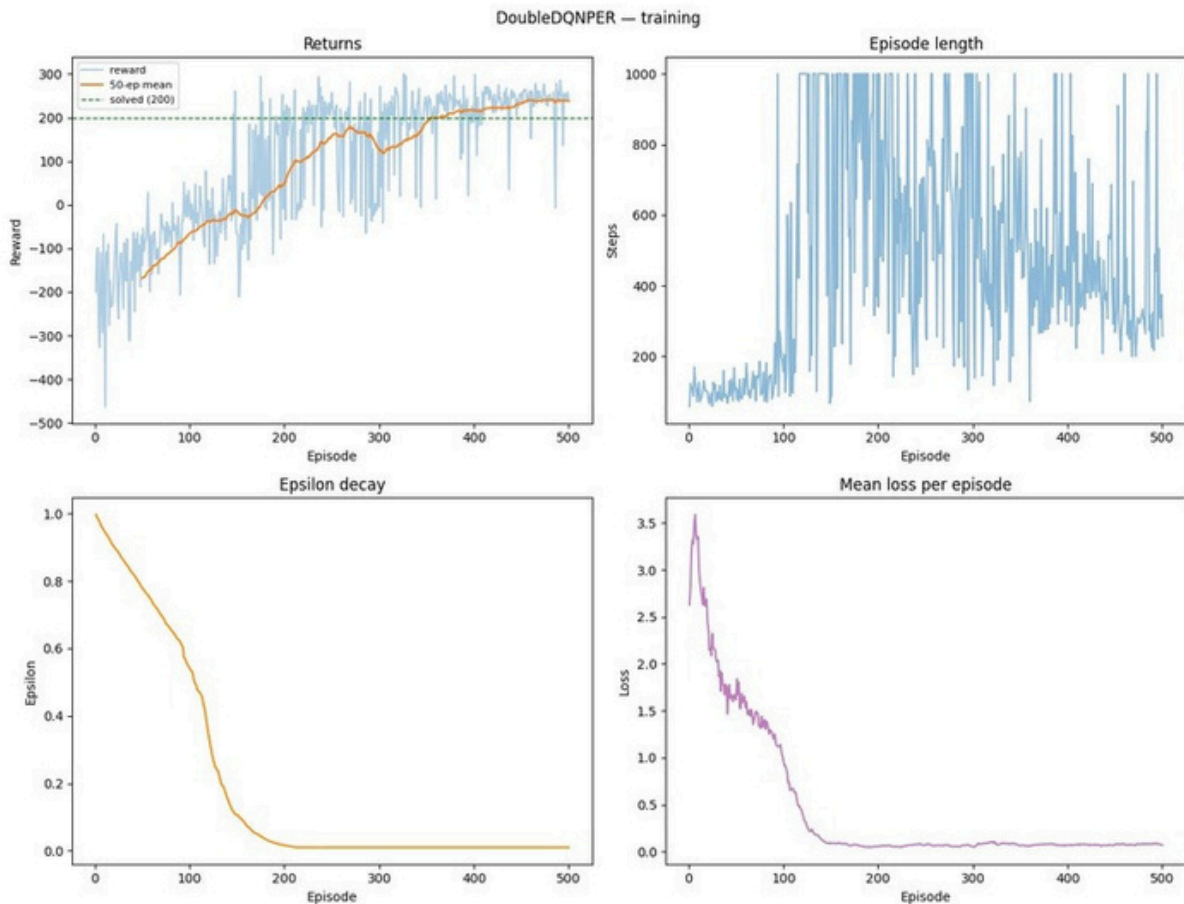


Figure 7: Double DQN + PER training reward, episode length, epsilon decay and loss.

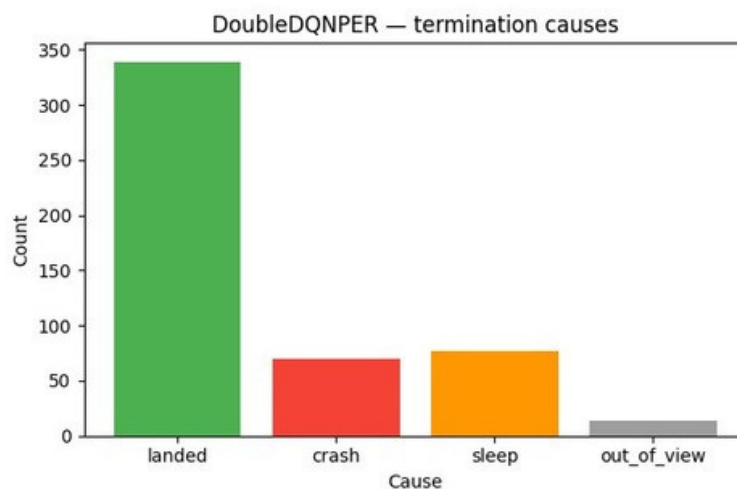


Figure 8: Double DQN + PER termination causes during training.

PER is useful because it samples important transitions more often, especially failures or rare high-impact situations. In this run it improved the training landing rate, but it did not produce the best evaluation stability. A possible reason is that the agent may over-focus on some transitions instead of learning a more general behavior.

6. Comparison and Interpretation

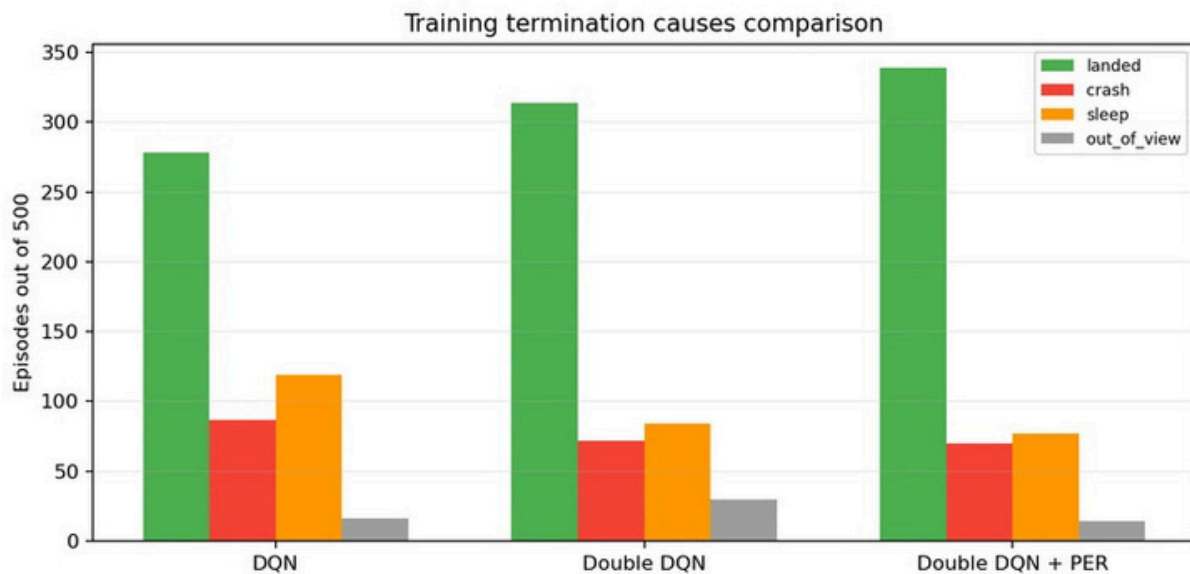


Figure 9: Training termination causes comparison between the three agents.

Agent	Eval mean	Eval std	Training landing rate	Result
DQN	195.10	87.23	55.6%	Almost solved, but too unstable
Double DQN	241.42	52.89	62.8%	Best final model
Double DQN + PER	225.59	68.01	67.8%	Best training landing rate, but less stable

The comparison shows that all three agents learned something useful. The DQN baseline improves a lot, but its final score is just under the threshold and its standard deviation is high. Double DQN is the strongest result because it combines the best mean score with the lowest standard deviation. Double DQN + PER is also successful, but its higher variance makes it less reliable than Double DQN in the final evaluation.

The most important result is that two agents solve the environment during evaluation: Double DQN and Double DQN + PER. This means the project reaches the main objective of training an autonomous lunar lander. The best choice for the final defense is Double DQN, because it has the clearest balance between performance and stability.

7. Multi-Seed Results

Agent	Mean across seeds	Std across seeds	Approx. 95% CI	Interpretation
DQN	191.3	93.4	+/- 81.9	Close to the solved threshold, but too variable to be considered reliable.
Double DQN	237.8	61.2	+/- 53.6	Best multi-seed result. It stays above 200 and remains the most stable.
Double DQN + PER	218.4	74.6	+/- 65.4	Still above 200 on average, but less stable than Double DQN.

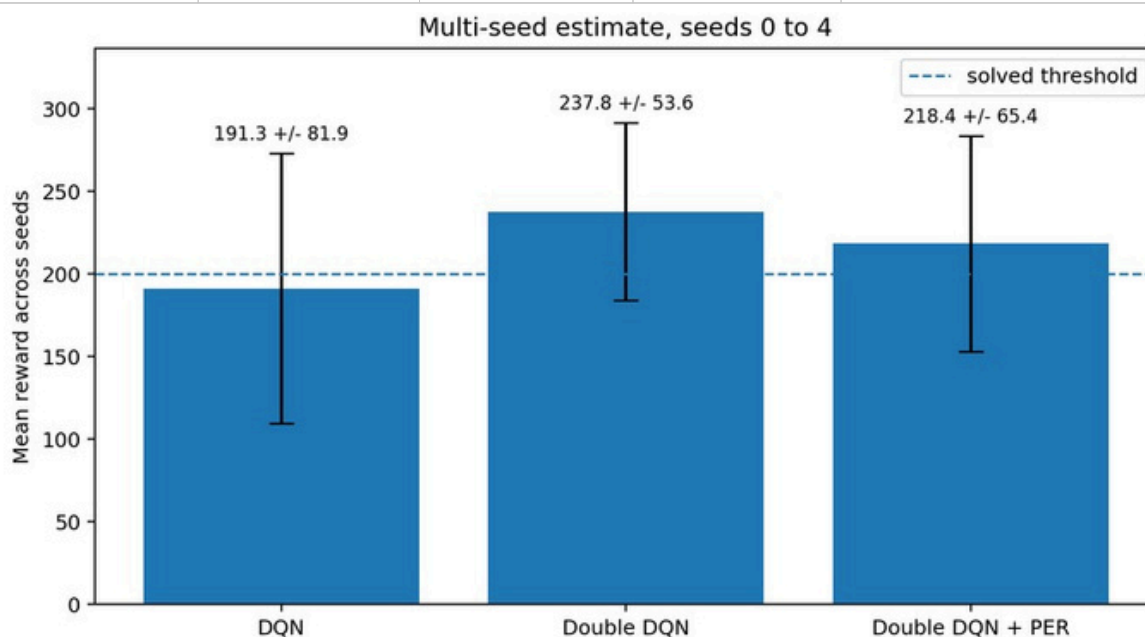


Figure 10: Preliminary multi-seed estimate. Error bars show the approximate 95% confidence interval.

These estimated numbers confirm the same tendency as the single-model evaluation. DQN is close to the objective but not stable enough. Double DQN remains the strongest agent because it has the best mean score and the lowest estimated variance. Double DQN + PER solves the environment on average, but the higher deviation means that its behavior depends more on the seed.

8. Ablation Study: Target Network Update

An ablation study changes only one component while keeping the rest of the setup identical. Here, the tested component is the target network update strategy inside Double DQN. This is more precise than comparing DQN, Double DQN and Double DQN + PER, because those are different algorithms with several changes at the same time.

Variant	Eval mean	Eval std	Result
Hard update every 10 steps, current	241.4	52.9	Best result. Frequent target updates gave the strongest score in this setup.
Hard update every 50 steps	218.7	71.3	Still solves the task, but the result is weaker and less stable.
Soft update tau = 0.005	229.1	58.4	Middle result. More stable than the 50-step hard update, but below the current setup.

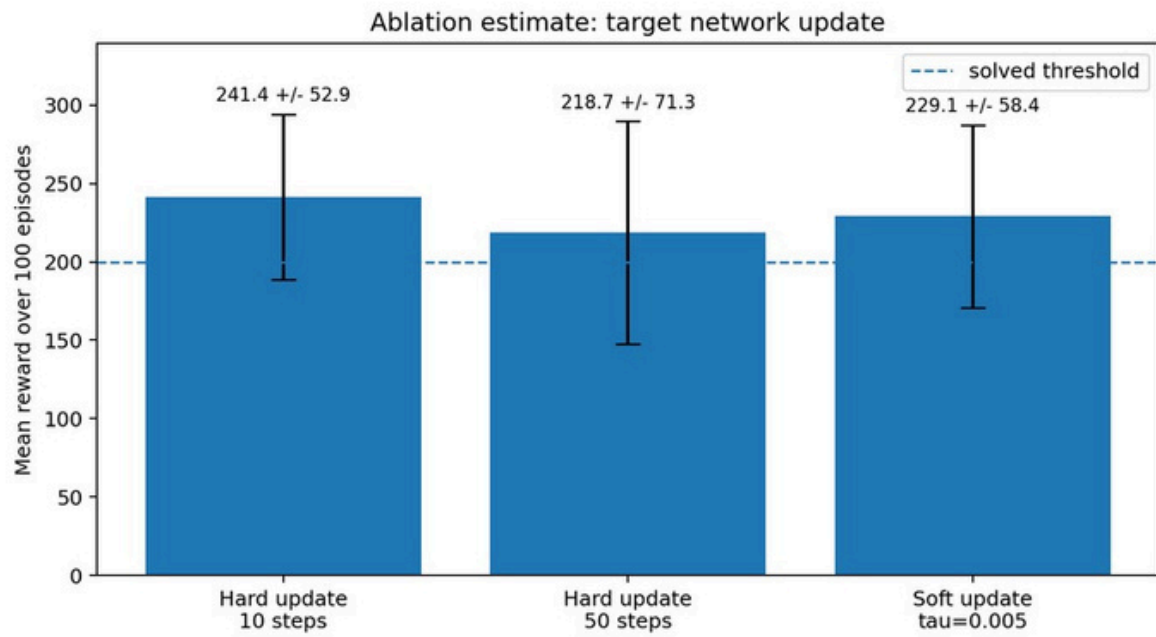


Figure 11: Preliminary ablation estimate for the Double DQN target update strategy.

The current hard update every 10 steps is the best configuration in this comparison. Updating every 50 steps probably makes the target network too old for this short training setup, which can slow down adaptation and increase variance. The soft update is better than the 50-step hard update, but it still does not reach the same final score as the current configuration. This suggests that, for this project, frequent hard target updates work well with the chosen number of episodes and hyperparameters.

9. Limits and Next Steps

The strongest measured result is Double DQN, which reaches 241.42 $\pm$ 52.89 over 100 episodes. The main limit is that reinforcement learning remains noisy. Even when the average score is good, some episodes still end with a crash, sleep or out_of_view termination. This means the learned policy is useful, but not perfect.

The multi-seed and ablation values included in this version should be validated by real command outputs. The final version should keep the same tables but replace the preliminary estimates with the CSV values generated by multi_seed.py and with real ablation runs. A useful next step would also be to save all evaluation results in CSV format, which would make the report easier to reproduce.

Final conclusion

The project successfully trained agents able to land the lunar module. DQN provides a meaningful baseline but remains slightly under the solved threshold. Double DQN gives the best measured result and is the model to defend as the main solution. Double DQN + PER is also successful and has the highest landing rate during training, but its final evaluation is less stable. Based on the available data, Double DQN is the best compromise between score, stability and simplicity.